R3.04 Qualité de développement :

Patrons de conception Adaptateur et Décorateur

Adaptateur

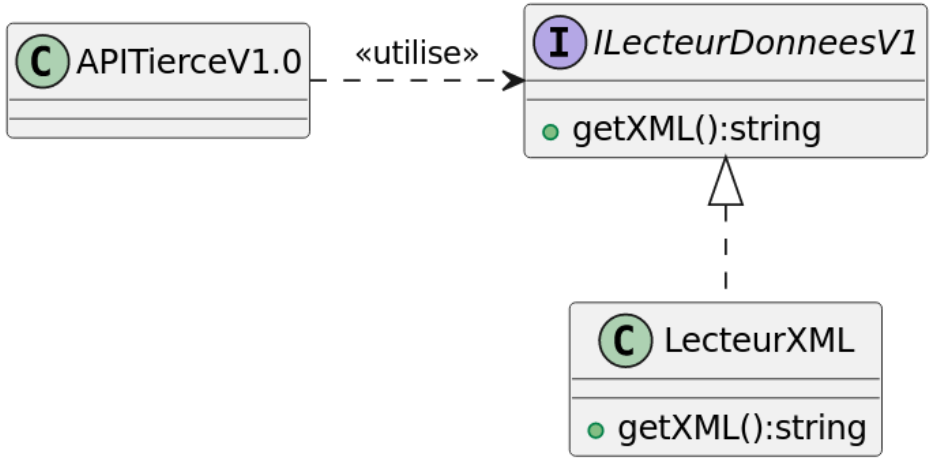
- GoF Pattern (Patron de structure)
- Question : comment interconnecter deux classes qui sont incompatibles ?
- Question : comment faire évoluer un programme qui utilise une bibliothèque dont l'interface a changé
- Exemple de bibliothèque utilisée
  - l'ancienne version attend du XML
  - la nouvelle version attend du JSON
  - → la classe qui produit du XML doit être **adaptée** pour produire du JSON

Deux types d'adaptateur

- Adaptateur de classe : nécessite l'héritage multiple
  - non supporté par beaucoup de langages (Java, TS, etc.)
- Adaptateur d'objets : utilise la composition

Adaptateur d'objets (1)

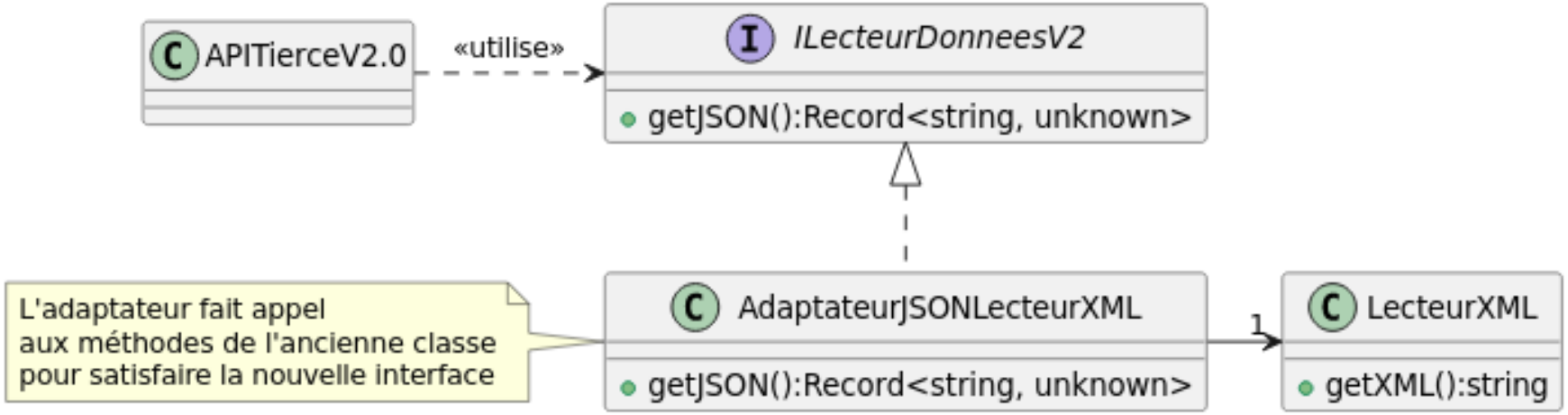
- Version 1 d'une bibliothèque tierce : utilise des données en XML
- Notre classe `LecteurXML` lui en fournit



- Mise à jour de la bibliothèque : utilise maintenant du JSON
- Comment faire ?

Adaptateur d'objets (2)

- Première possibilité
  - ajouter une méthode de conversion à la classe LecteurXML
  - inconvénient : on modifie du code qui fonctionne ; la classe a maintenant deux responsabilités
- Seconde possibilité : envelopper la classe `LecteurXML` dans un adaptateur qui se charge de la conversion



Adaptateur d'objets (3)

- Exemple de code de l'adaptateur

```
import { xml2js } from "https://deno.land/x/xml2js@1.0.0/mod.ts";
import { ILecteurDonnees } from "./ILecteurDonneesV2.ts";
import { LecteurXML } from "./LecteurXML.ts";

export class AdaptateurJSONLecteurXML implements ILecteurDonneesV2 {

  private _lecteurXML: LecteurXML;

  public constructor(lecteurXML: LecteurXML) {
    this._lecteurXML = lecteurXML;
  }

  public getJSON(): Record<string, unknown> {

    const xml = this._lecteurXML.getXML();
    return xml2js(xml, { compact: true });
  }
}
```

Adaptateur d'objets (4)

- L'API tierce ne sait pas qu'elle utilise un objet adapté

```
class APITierceV20 {
  public constructor(fournisseurDonnees: ILecteurDonneesV2) {
    console.log(fournisseurDonnees.getJSON());
  }

  ...
}
```

- Instanciation de l'API tierce avec l'objet adapté

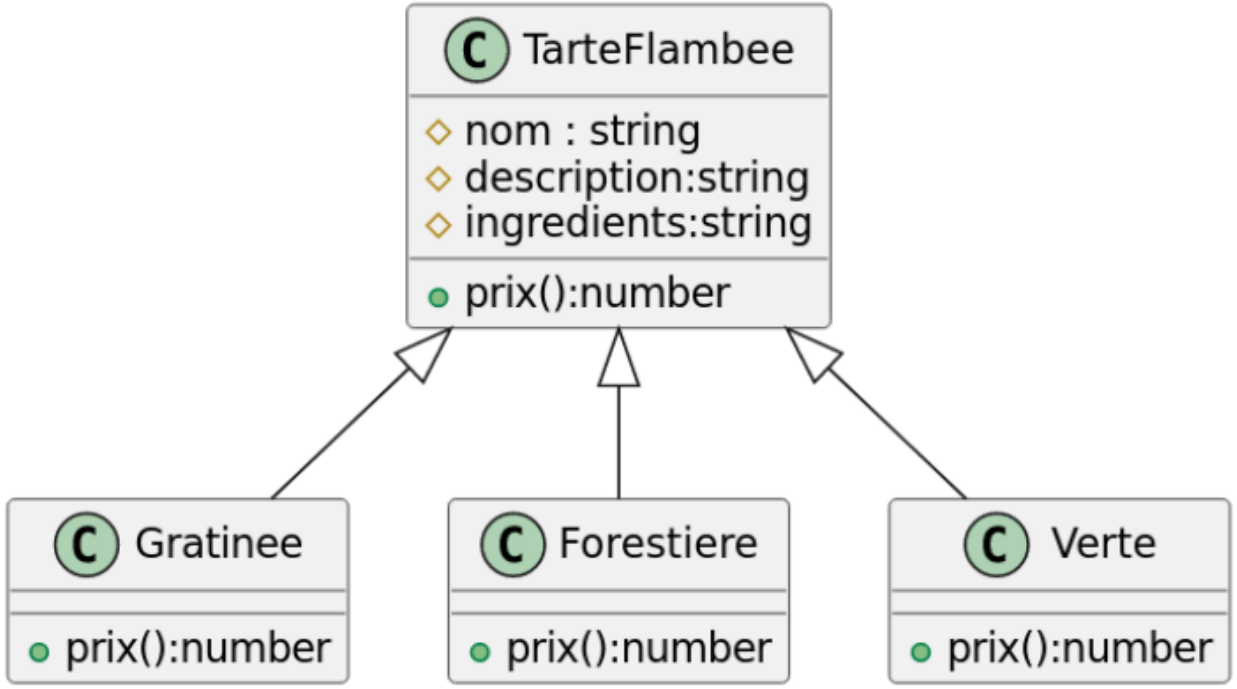
```
new APITierceV20(new AdaptateurJSONLecteurXML(new LecteurXML()));
```

Décorateur

- GoF Pattern (Patron de structure)
- Question : comment ajouter dynamiquement des comportements à un objet
- Possibilité de créer de nouveaux objets sans avoir explicitement prévu leur structure
- Alternative à l'héritage.

Décorateur - exemple

- Gestion d'un menu de restaurant



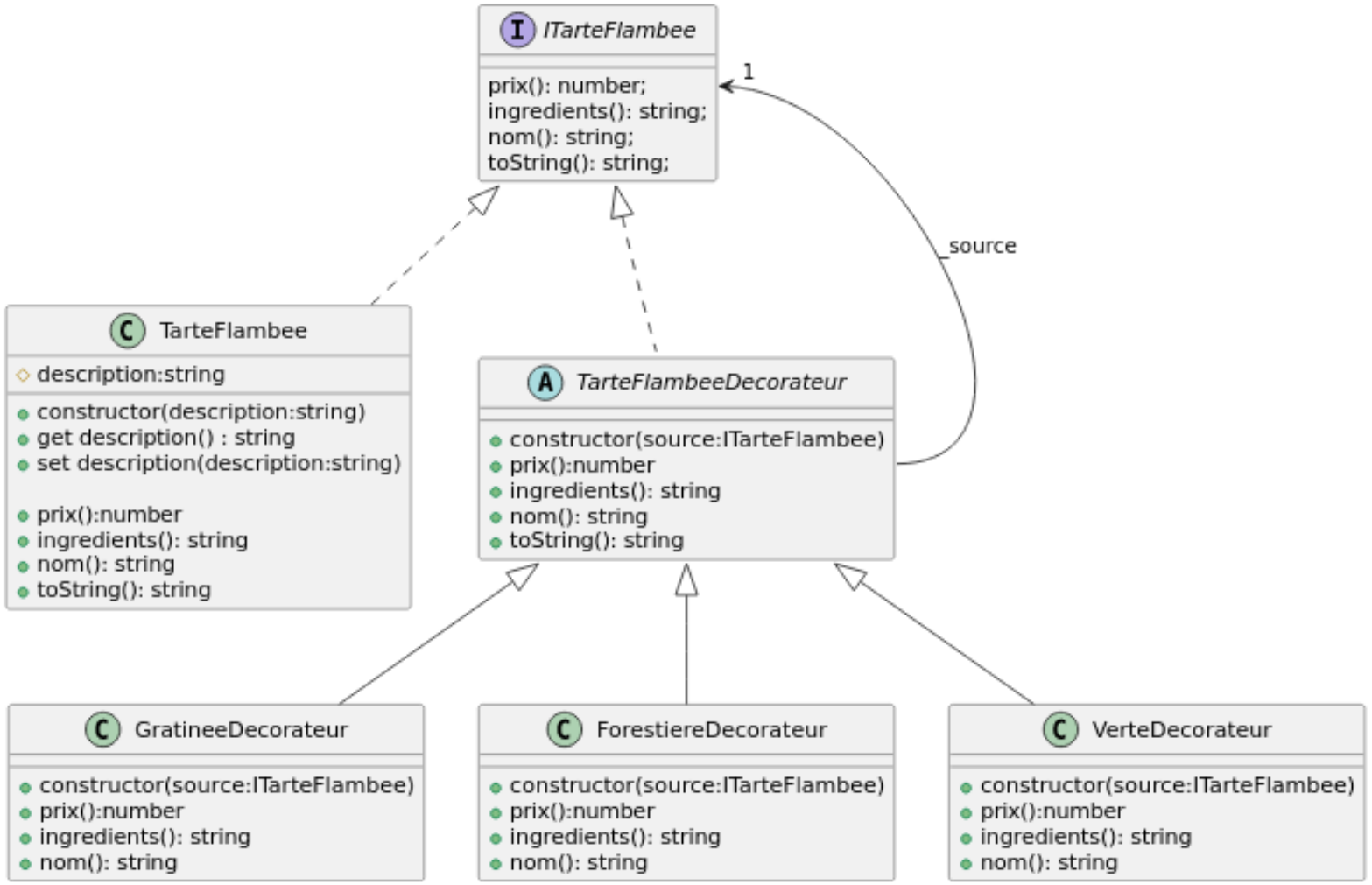
Décorateur - exemple

- Comment prévoir des évolutions ?
  - Forestière gratinée
  - Saumon Ciboulette
  - Saumon Ciboulette Gratinée
  - Double Forectière Saumon
  - etc.
- Potentiellement il y a un nombre exponentiel de classes à écrire

Décorateur - procédé

- l'idée du décorateur est d'emballer des objets à l'aide d'autres objets qui possèdent la même interface
- Une forestière gratinée c'est:
  - une tarte flambée
  - décorée avec des champignons
  - décorée avec du Comté râpé
- En utilisant ce principe, on peut créer des objets à l'infini

Decorateur - diagramme de classes



Décorateur - code (1)

Extraits de la classe `TarteFlambee`

```
export class TarteFlambee implements ITarteFlambee {

  private _description!: string;

  constructor() {
    this.description =
      "Délicieuse spécialité Alsacienne à base de crème, oignons et lardons.";
  }
  public prix(): number {
    return 10;
  }
  public nom(): string {
    return "Classique";
  }
  public ingredients(): string {
    return "Crème, oignons, lardons";
  }
  public toString(): string {
    return this.nom() + " (" + this.ingredients() + ") : " + this.prix() + "€";
  }
```

Décorateur - code (2)

Décorateur abstrait, qui va permettre d'emballer :

- un autre décorateur
- la classe concrète `TarteFlambee`

```
export abstract class TarteFlambeeDecorateur implements ITarteFlambee {
  protected _source: ITarteFlambee;

  constructor(source: ITarteFlambee) {
    this._source = source;
  }

  abstract prix(): number;
  abstract ingredients(): string;
  abstract nom(): string;

  public toString(): string {
    return this.nom() + " (" + this.ingredients() + ") : " + this.prix() + "€";
  }
}
```

Décorateur - code (3)

Décorateur concret : 10% plus cher, les ingrédients de l'objet emballé + fromage, nom de l'objet emballé + " Gratinée"

```
export class GratineeDecorateur extends TarteFlambeeDecorateur {
  constructor(source: ITarteFlambee) {
    super(source);
  }

  public prix(): number {
    return 1.1 * this._source.prix();
  }

  public ingredients(): string {
    return this._source.ingredients() + ", Comté gratiné";
  }

  public nom(): string {
    return this._source.nom().replace("Classique", "") + " Gratinée";
  }
}
```

Decorateur - code (4)

Utilisation

```
const tt = new TarteFlambee();
console.log(tt.toString());
// Classique (Crème, oignons, lardons) : 10€

const ttg = new GratineeDecorateur(tt);
console.log(ttg.toString());
// Gratinée (Crème, oignons, lardons, Comté gratiné) : 11€

const ttgvf = new GratineeDecorateur(
        new CibouletteDecorateur(new ChampignonsDecorateur(tt))
console.log(ttgvf.toString());
// Forestière Verte Gratinée (Crème, oignons, lardons,
// champignons, ciboulette, Comté gratiné) : 13.86€
```

Décorateur - remarque

- Pour ce qu'on vient de faire, ce qui suit est suffisant :

```
private _types : Map<string, [string, number]> = new Map([
    ["Classique", ["Crème, oignons, lardons", 10]],
    ["Munster", ["Crème, oignons, lardons, munster", 12]],
    ["Forestière", ["Crème, oignons, lardons, champignons", 11]],
    ["Végétarienne", ["Crème, oignons, champignons, poivrons, tomates", 14]],
    ["Chèvre", ["Crème, oignons, lardons, chèvre", 11]],
    ["Saumon", ["Crème, oignons, saumon", 12]]
  ]);
```

- Le patron de conception *décorateur* permet d'ajouter, dans chaque décorateur, d'autres attributs et d'autres méthodes